ECR-GRPO: Event-Conditioned Credit Refill as a GRPO Plug-in for Asynchronous Long-Horizon LLM Agents

Anonymous submission

Abstract

Long-horizon LLM agents act through extended sequences of tool calls, environment observations, and intermediate decisions. In such settings, supervision rarely arrives as an immediate scalar reward attached to the action that caused it. A tool result may be observed several steps late; a terminal evaluator may judge the whole episode only after the relevant choice has passed; partial feedback may be dropped by timeouts or interruptions. Standard GRPO pipelines can optimize a policy from scalar rewards, but they do not specify how asynchronous events are converted into the reward consumed by GRPO. This paper introduces ECR-GRPO, a reward-construction plug-in for GRPO rather than a replacement optimizer. ECR-GRPO keeps recent steps in a pending buffer, represents feedback as asynchronous events, and refills step returns when events arrive. A no-oracle evidence kernel scores event-step affinity using only public trace evidence, including temporal distance, action text, tool metadata, public tags, and observation changes. The resulting rewards are passed to the standard GRPO interface; group-relative normalization, ratio clipping, entropy terms, and policy updates are unchanged. Controlled asynchronous diagnostics show that evidence-based refill improves both task performance and credit localization over terminal, uniform, and recency reward sources under the same GRPO update. A transfer study on parameter-efficiently fine-tuned language-model policies further shows that event-gated routing preserves local learning while correcting non-local credit assignment.

Introduction

Asynchronous feedback is common in agentic RL

LLM agents increasingly solve tasks through interaction rather than single-turn generation: they query tools, browse or search, edit code, call APIs, and act in text-based embodied environments (Yao et al. 2022b,a; Shridhar et al. 2020). These traces create a supervision pattern that is poorly matched to the synchronous reward abstraction. Feedback may arrive after the action that caused it, may describe an earlier subgoal rather than the latest step, or may be missing because an environment timed out. The learning problem is therefore not only how to optimize long trajectories, but how to assign delayed event feedback to the decisions that made it relevant.

Trajectory-level reinforcement learning treats this assignment coarsely. A failed episode can contain useful search, extraction, and tool-use steps before a late formatting error or wrong final argument. Broadcasting the terminal failure across the whole trajectory punishes those useful steps together with the actual error. The reverse problem also occurs: a delayed positive event can support an earlier action, while a recency heuristic assigns it to the latest step. Long-horizon agent training needs a credit mechanism that respects both temporal order and semantic evidence.

Reward granularity and reward timing are different issues

PPO and GRPO-style training provide effective policy optimization objectives for language models and reasoning systems (Schulman et al. 2017; Shao et al. 2024; DeepSeek-AI 2025; Zhou et al. 2026). Process supervision and verifier-based training improve feedback granularity by judging intermediate reasoning or steps (Cobbe et al. 2021; Lightman et al. 2023). These advances address an important part of the problem, but they do not by themselves solve asynchronous feedback. Step-level optimization is only reliable when the reward attached to a step is already known. In deployed agents, feedback often first appears as an event stream: tool outputs, observation deltas, partial rewards, timeout penalties, terminal judgments, and non-local support signals.

ECR-GRPO separates two questions that are often collapsed. The first is the optimization question of how a policy update uses scalar rewards. The second is the credit construction question of how asynchronous feedback is converted into those rewards. This paper focuses on the second question. The downstream GRPO objective is not modified.

Event-conditioned credit refill

ECR-GRPO records each action as a pending step. When an asynchronous event arrives, a credit kernel selects eligible historical steps and assigns event reward back into their returns. The main kernel does not use oracle links such as exact related step identifiers. Instead, it scores candidate steps using weak evidence available in realistic traces: temporal proximity, action text, tool names, public tags, and observation changes. The output is a GRPO-consumable reward

sample. All subsequent group-relative advantage normalization and policy updates follow the standard GRPO pipeline.

The method also introduces event-gated routing. Local feedback is kept sharp because it is often most useful for next-action learning. Non-local support events, in contrast, are routed through evidence-based attribution. Gating lets the same training loop handle both cases without forcing every event into a single credit heuristic.

Contributions

This paper makes four contributions.

1. It formulates asynchronous step credit assignment for long-horizon LLM agents, where feedback arrives as delayed, missing, interrupted, or non-local events.
2. It introduces Event-Conditioned Credit Refill, a mechanism that keeps steps pending and refills their returns when events arrive.
3. It proposes a no-oracle evidence attribution kernel and an event-gated routing rule that separates local feedback from non-local support events.
4. It evaluates the mechanism as a reward-construction module under a fixed GRPO optimizer, reporting both task performance and credit-localization metrics.

Background and related work

Group-relative policy optimization

Policy-gradient methods such as PPO optimize a clipped surrogate objective from sampled rollouts (Schulman et al. 2017). GRPO-style training replaces value estimation with group-relative normalization, which has made it attractive for reasoning-oriented language-model training (Shao et al. 2024; DeepSeek-AI 2025). The analysis of GRPO as a group-relative policy-gradient estimator further clarifies how group statistics shape the update (Zhou et al. 2026). ECR-GRPO keeps this optimization interface intact. Its change is upstream: before standard GRPO receives rewards, asynchronous events are refilled into step or trajectory returns.

Process and step supervision

Verifier and process-reward methods show that intermediate supervision can improve learning when final answers are too coarse (Cobbe et al. 2021; Lightman et al. 2023). In agent settings, intermediate actions and tool calls offer similar opportunities for fine-grained training. The distinction in this work is that step-level supervision is not assumed to be pre-aligned. ECR-GRPO models the arrival process explicitly and estimates which historical steps receive credit when feedback is delayed or non-local.

Long-horizon language agents

Language agents combine reasoning, acting, search, and environment interaction (Yao et al. 2022b; Shinn et al. 2023; Zhou et al. 2023). Benchmarks such as WebShop and ALFWorld expose sparse outcomes and multi-step dependencies (Yao et al. 2022a; Shridhar et al. 2020). Recent agent-training systems also emphasize the need to train agents from interaction traces rather than isolated completions (Luo et al. 2025). These settings motivate credit assignment mechanisms that operate on logged event streams and agent traces rather than on a single terminal scalar.

Delayed reward and semantic credit

Delayed-reward methods such as RUDDER decompose returns to address temporal credit assignment (Arjona-Medina et al. 2018). LLM agents add a second source of evidence: trace semantics. Actions contain text, tool names, arguments, public tags, and observation changes. ECR-GRPO uses these signals to distinguish the latest step from the step that is semantically related to a later event. The resulting attribution is intentionally transparent, so performance gains can be inspected through credit metrics rather than only through task reward.

Problem setting

Step records and asynchronous events

At environment time t, the agent produces a step record

$$b_t = (\text{task}, \text{episode}, \text{group}, t, o_t, a_t, \ell_t, \mathcal{A}_t, m_t),$$

where o_t is the observation, a_t is the selected action, ℓ_t is the old log probability used for policy updates, $\mathcal{A}_t$ is the candidate action set when available, and m_t contains public trace metadata.

Feedback is represented as an event

$$e_k = (\text{task}, \text{episode}, k, \tau_k, R_k, \Delta o_k, y_k, \mu_k),$$

where τ_k is the event time, R_k is the reward carried by the event, Δo_k is an observation change or textual feedback payload, y_k denotes event type, and μ_k contains public event metadata. Events can encode terminal outcomes, partial feedback, tool returns, timeout penalties, interruption penalties, and non-local support signals. Missing feedback is modeled by dropping or withholding events rather than by inventing a separate reward type.

Credit refill objective

When event e_k arrives, ECR-GRPO assigns its reward to eligible historical steps from the same task and episode. For each eligible step b_t, the refill credit is

$$c_{t,k} = w_{t,k} R_k, \qquad \sum_{t \in \mathcal{B}(e_k)} w_{t,k} = 1,$$

where $\mathcal{B}(e_k)$ is the pending candidate set. The refilled step return is

$$G_t = r_t + \sum_k c_{t,k}.$$

Here r_t denotes the synchronous immediate reward observed at the action boundary, while $c_{t,k}$ denotes credit refilled from subsequently arriving asynchronous events. The two sources are non-overlapping: r_t covers feedback available immediately after action execution, and $c_{t,k}$ covers delayed event feedback.

The central modeling problem is the choice of $w_{t,k}$. A terminal-reward GRPO source spreads event reward over an entire episode. A recency-refilled GRPO source assigns more credit to recent steps. ECR-GRPO instead estimates event-step affinity from public trace evidence before handing the resulting reward to GRPO.

No-oracle attribution

The deployable setting does not expose exact causal links such as `related_step_id`, `related_tool`, or `related_subgoal`. Such fields may be logged for diagnostics, but they are stripped before the credit kernel runs. This no-oracle protocol matters because an attribution method that relies on exact links is mostly a measurement tool, not a training mechanism. The Dependency kernel in the experiments is therefore treated as an oracle upper bound, not as a deployable method.

Method

Pending step buffer

Each executed step enters a pending buffer. When an event arrives, the buffer selects steps from the same episode whose environment time precedes the event. Steps remain eligible until terminal finalization or pending-window expiration. The buffer converts a stream of actions and delayed events into a local assignment problem: for this event, among these still-eligible steps, which step or set of steps receives the event reward?

Evidence-conditioned attribution

The evidence kernel assigns weights by scoring event-step affinity:

$$\begin{aligned} K(e_k, b_t) &= \alpha_{\text{time}} s_{\text{time}}(e_k, b_t) + \alpha_{\text{text}} s_{\text{text}}(e_k, b_t) \\ &\quad + \alpha_{\text{tool}} s_{\text{tool}}(e_k, b_t) + \alpha_{\text{tag}} s_{\text{tag}}(e_k, b_t) \\ &\quad + \alpha_{\Delta} s_{\Delta}(e_k, b_t), \\ w_{t,k} &= \frac{\exp(K(e_k, b_t)/T)}{\sum_j \exp(K(e_k, b_j)/T)}, \quad b_j \in \mathcal{B}(e_k). \end{aligned}$$

The score combines temporal distance, lexical overlap between event text and action text, tool or API matches, public tag overlap, and observation-delta matches. The kernel records the assigned weight, entropy, top margin, selected route, and an explanation string for each event. These diagnostics make it possible to separate better learning from merely larger reward magnitude.

Event-gated routing

Not all events call for the same attribution rule. Local feedback is often tied to the immediately preceding action and is kept sharp. Non-local support events are precisely the cases where recency is misleading. ECR-GRPO uses a deterministic delay gate rather than a hand-labeled event switch. For event e_k, define the observed delay against the current pending set as

$$\Delta_k = \tau_k - \max\{t \mid b_t \in \mathcal{B}(e_k)\}.$$

Algorithm 1: Event-conditioned credit refill

for each rollout group **do**
  for each episode **do**
    execute actions and append each b_t to the pending buffer
    for each arriving event e_k **do**
      select eligible pending steps $\mathcal{B}(e_k)$
      route e_k to a credit kernel
      compute weights $w_{t,k}$ and refill $c_{t,k} = w_{t,k} R_k$
      finalize terminal or expired steps
    end for
  end for
  emit GRPO samples; standard GRPO normalizes advantages and updates the policy
end for

The gate $g(e_k)$ selects the credit rule:

$$g(e_k) = \begin{cases} \text{Recency}, & \Delta_k \le \delta,\ y_k \notin \mathcal{Y}_T, \\ \text{Evidence}, & \Delta_k > \delta,\ y_k \notin \mathcal{Y}_T, \\ \text{Mix}_\alpha, & y_k \in \mathcal{Y}_T. \end{cases}$$

Here δ is an environment-level delay threshold, α is a terminal-event mixing coefficient, $\mathcal{Y}_T$ contains terminal success and terminal failure events, and Mix_α denotes the convex combination $\alpha\,\text{Recency} + (1-\alpha)\,\text{Evidence}$. The selected rule produces $w_{t,k}$. Ambiguous events are routed by the same delay rule, which makes the gate reproducible and gives the experiments a direct sensitivity parameter.

This routing is especially important for language-model policies. Pure evidence attribution can improve non-local credit placement but dilute local next-action learning. Gated Evidence preserves the local signal while still correcting delayed support events.

GRPO-compatible update

After refill, the adapter emits GRPO samples with prompt, completion, group identifier, and reward. For step-level samples, the reward is G_t. For trajectory-level samples, the reward is the sum of refilled returns along the episode. Standard GRPO then normalizes rewards within rollout groups:

$$A_t = \frac{G_t - \text{mean}(G_{\text{group}})}{\text{std}(G_{\text{group}}) + \epsilon}.$$

The policy update uses the same clipped ratio form as PPO/GRPO-style optimization. ECR-GRPO therefore changes the construction of G_t, not the optimizer interface.

Empirical variance-reduction intuition

Exact unbiasedness would require oracle access to the true causal links between events and steps. ECR-GRPO does not assume such access. Its no-oracle evidence kernel is a semantically consistent heuristic: it concentrates reward on historical steps whose public trace evidence matches the arriving event, instead of spreading the same signal uniformly across the trajectory. This does not change the GRPO objective, but

it can reduce gradient noise because fewer irrelevant steps receive event reward mass. The controlled benchmark tests this intuition empirically through credit-localization metrics, including target weight, recent-step weight, and argmax target rate.

Experimental design

Controlled asynchronous diagnostics

The controlled benchmark creates long-horizon text-action tasks with known target actions and delayed support events. This setting exposes ground-truth target steps for post-hoc credit evaluation while still withholding oracle links from the training kernel. It is used to test whether a method learns better because it assigns delayed reward to better historical steps.

LLM policy transfer with LoRA

We implement the agent policy as a causal language model using the Hugging Face (HF) Transformers library and conduct parameter-efficient fine-tuning with Low-Rank Adaptation (LoRA). Candidate actions are scored from the prompt, normalized into an action distribution, and updated through selected-action log probabilities with clipped GRPO/PPO-style ratios. This experiment tests whether the credit construction mechanism transfers from a diagnostic policy to a realistic LLM agent training loop.

External-validity stress protocol

The same event interface applies to ALFWorld-style environments, where agents act through admissible text actions and sparse terminal success (Shridhar et al. 2020). The stress protocol separates two regimes: native environment interaction and async-perturbed interaction. The perturbations delay terminal feedback, drop partial feedback, inject timeout and interruption events, and preserve the same no-oracle separation used in the controlled benchmark. This protocol tests the interface under realistic long-horizon interaction without changing the credit-assignment abstraction.

Baselines and metrics

The comparison isolates reward construction. All methods use the same GRPO optimizer, tasks, rollout budgets, action spaces, and policy families where applicable:

1. GRPO-Terminal: terminal reward is assigned at the rollout level.
2. GRPO-Uniform: terminal reward is spread uniformly across episode steps.
3. GRPO-Recency: event credit decays with temporal distance.
4. GRPO-Evidence: no-oracle evidence-conditioned attribution.
5. GRPO-Gated: non-terminal events are routed by the delay gate; short-delay events use recency, long-delay events use evidence, and terminal events use the mixture in Section 4.3.
6. Dependency Oracle: exact-link attribution used only as an upper-bound diagnostic.

Table 1: Controlled asynchronous diagnostic performance.

Method	Success	Return
GRPO-Terminal	0.000	-0.289
GRPO-Uniform	0.383	0.338
GRPO-Recency	0.675	0.474
GRPO-Evidence	0.725	0.501

Table 2: Credit-localization diagnostics for delayed non-local events.

Method	Target weight	Recent weight	Argmax target
Evidence	0.418	0.176	0.977
Recency	0.132	0.347	0.000
Uniform	0.188	0.166	0.441

Task metrics include success rate, average return, learning AUC, and steps to success where applicable. Credit metrics include target weight, recent-step weight, target credit fraction, argmax target rate, top-3 target rate, attribution entropy, and top margin.

Results

Controlled task performance

Table 1 shows that terminal-reward GRPO fails in the controlled asynchronous setting. Uniform refill recovers part of the signal by moving reward from episode level to step level. Recency is a strong reward source when delay is short, but GRPO-Evidence performs best because it can place delayed non-local reward on semantically related historical actions.

Credit localization

The task gains in Table 1 correspond to better credit placement. Table 2 reports diagnostic attribution metrics under the same controlled setting. Evidence assigns substantially more weight to the target historical step and sharply reduces weight on the most recent step. Recency learns useful behavior but does so with a systematic bias toward the latest action.

LLM policy transfer with LoRA

Table 3 shows the effect of credit routing in the language-model transfer setting. GRPO-Recency and GRPO-Gated reach the same final success, but GRPO-Gated obtains higher logged mean success and much better non-local attribution. GRPO-Evidence assigns delayed support events accurately, but its lower local-learning signal reduces final task performance. The result supports event-conditioned routing: local feedback and non-local support events benefit from different credit rules.

Mechanism analysis

The controlled and LLM policy-transfer results point to the same mechanism. Evidence changes where delayed reward is assigned, not simply how much reward the agent receives. This distinction matters because long-horizon agents often

Table 3: LLM policy transfer with LoRA and candidate-action scoring.

Method	Final	Mean	Target	Recent	Argmax
GRPO-Rec.	0.917	0.587	0.195	0.307	0.000
GRPO-Evid.	0.889	0.540	0.623	0.095	1.000
GRPO-Gated	0.917	0.658	0.755	0.080	1.000

contain useful intermediate actions inside failed episodes. The credit-localization metrics show whether a method rewards those actions or collapses to the latest step before feedback. Gated Evidence is the strongest variant in the language-model setting because it keeps local feedback local while sending non-local support through semantic attribution.

Discussion

Why evidence helps

Temporal proximity is useful but incomplete. A late event often contains text or metadata that points to an earlier action: a tool name, entity mention, subgoal tag, or observation change. Evidence attribution uses those signals to avoid assigning delayed feedback blindly to the most recent step. This is most valuable when the event describes the consequence of an earlier decision rather than the state of the latest action.

Why gating helps

Evidence is not always the right default. Local partial feedback is often meant to shape the next action, and spreading it across several semantically related steps can weaken learning. Gating makes the credit rule conditional on observed delay and terminal status. The method therefore treats local corrective feedback and delayed support feedback as different learning signals, even though both arrive through the same event interface.

Relation to step-level optimization

ECR-GRPO is complementary to step-level optimization. Step-level methods change the unit of optimization; ECR-GRPO constructs the rewards consumed by that optimization. In practice, the credit refill module can feed standard GRPO, a GRPO variant, or a stronger step-level objective. The contribution is the event-to-step assignment layer that sits before the policy update.

Limitations

The method assumes that delayed events carry recoverable trace evidence. When actions are repetitive, metadata is sparse, event text is uninformative, or feedback arrives after the pending window expires, the evidence kernel has less signal to separate the relevant step from nearby alternatives. The scorer used in this paper is deliberately transparent and rule-based. A learned scorer could capture richer semantic relations, but it would also make attribution harder to audit. The strongest evidence reported here comes from settings where ground-truth credit can be measured, which is appropriate for diagnosing the mechanism but narrower than the full range of deployed agent traces.

Conclusion

ECR-GRPO treats delayed agent feedback as an event-to-step credit assignment problem. By keeping steps pending and refilling their returns when asynchronous events arrive, the method converts delayed and non-local supervision into rewards that standard GRPO can consume. The optimizer remains unchanged. The results show that evidence-based refill improves credit localization and task performance in controlled asynchronous diagnostics, while event-gated routing is the more reliable reward source for language-model policy transfer. These findings support a separation between policy optimization and feedback assignment: long-horizon LLM agents need an explicit mechanism for deciding which prior step an event trains.

References

Arjona-Medina, J. A.; Gillhofer, M.; Widrich, M.; Unterthiner, T.; Brandstetter, J.; and Hochreiter, S. 2018. RUDDER: Return Decomposition for Delayed Rewards. *arXiv*.

Cobbe, K.; Kosaraju, V.; Bavarian, M.; Chen, M.; Jun, H.; Kaiser, L.; Plappert, M.; Tworek, J.; Hilton, J.; Nakano, R.; Hesse, C.; and Schulman, J. 2021. Training Verifiers to Solve Math Word Problems. *arXiv*.

DeepSeek-AI. 2025. DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning. *arXiv*.

Lightman, H.; Kosaraju, V.; Burda, Y.; Edwards, H.; Baker, B.; Lee, T.; Leike, J.; Schulman, J.; Sutskever, I.; and Cobbe, K. 2023. Let's Verify Step by Step. *arXiv*.

Luo, X.; Zhang, Y.; He, Z.; Wang, Z.; Zhao, S.; Li, D.; Qiu, L. K.; and Yang, Y. 2025. Agent Lightning: Train ANY AI Agents with Reinforcement Learning. *arXiv*.

Schulman, J.; Wolski, F.; Dhariwal, P.; Radford, A.; and Klimov, O. 2017. Proximal Policy Optimization Algorithms. *arXiv*.

Shao, Z.; Wang, P.; Zhu, Q.; Xu, R.; Song, J.; Bi, X.; Zhang, H.; Zhang, M.; Li, Y. K.; Wu, Y.; and Guo, D. 2024. DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models. *arXiv*.

Shinn, N.; Cassano, F.; Berman, E.; Gopinath, A.; Narasimhan, K.; and Yao, S. 2023. Reflexion: Language Agents with Verbal Reinforcement Learning. *arXiv*.

Shridhar, M.; Yuan, X.; Cote, M.-A.; Bisk, Y.; Trischler, A.; and Hausknecht, M. 2020. ALFWorld: Aligning Text and Embodied Environments for Interactive Learning. *arXiv*.

Yao, S.; Chen, H.; Yang, J.; and Narasimhan, K. 2022a. WebShop: Towards Scalable Real-World Web Interaction with Grounded Language Agents. *arXiv*.

Yao, S.; Zhao, J.; Yu, D.; Du, N.; Shafran, I.; Narasimhan, K.; and Cao, Y. 2022b. ReAct: Synergizing Reasoning and Acting in Language Models. *arXiv*.

Zhou, A.; Yan, K.; Shlapentokh-Rothman, M.; Wang, H.; and Wang, Y.-X. 2023. Language Agent Tree Search Unifies Reasoning Acting and Planning in Language Models. *arXiv*.

Zhou, H.; Ye, K.; Xu, E.; Zhu, J.; Gong, S.; and Shi, C. 2026. Demystifying Group Relative Policy Optimization: Its Policy Gradient is a U-Statistic. *arXiv*.